toggle. By itself this doesn't do much. But when the page is loaded, Stimulus will look for a matching controller. Using convention over configuration, that controller should be in the file app/javascript/controllers/favorite_toggle_controller.ts.

Here's a basic controller that doesn't do anything yet. After creating this controller file, we need to call ./bin/rails stimulus:manifest:update to update the manifest file. (We could use the generator command referenced in the file comment, but that will create a .js file, not a .ts file.)

chapter_03/01/app/javascript/controllers/favorite_toggle_controller.ts

```
import { Controller } from "@hotwired/stimulus";

export default class FavoriteToggleController extends Controller {
  connect(): void {
    console.log("The controller is connected");
  }
}
```

The first line uses ES6 module syntax to import the Controller class from the @hotwire/stimulus module. Because we are using esbuild, this import statement is reconciled against the Stimulus module living in our node_modules directory.

Next we declare our class, again using ES6 module keywords. The export keyword means that the thing about to be defined is publicly visible to other files that might import this file. The default keyword means the thing about to be defined is the default item exported if the importing module does not specify what it wants. The actual class name is optional here, since it will be inferred by Stimulus from the file name, but I find it easier to have the name spelled out explicitly.

Inside the class, we define one method: connect(): void. Don't get too attached to it, because we're not going to keep it. The connect() method is automatically called by Stimulus when a controller is instantiated and connected to a DOM element. I included it here only to allow you to see that the controllers are, in fact, instantiated when the page is reloaded (we can actually learn the same thing by setting application.debug = true in the controller/application.js file).